

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
AI(Artificial Intelligence)
Practical VII Part 2 & 3

Roll No.:T031	Name:Ishita Patel
Class:TYIT	Batch:01
Date of Assignment: 5-08-2026	Date/Time of Submission: 5-08-2026

2. Using a suitable regression dataset, implement Linear Regression using scikit-learn.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Load dataset
data = pd.read_csv("student_marks.csv")
# Input and output
X = data[['Hours']]
y = data['Marks']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:",
      round(model.score(X_train, y_train) * 100, 2))

print("Testing Accuracy:",
      round(model.score(X_test, y_test) * 100, 2))
# Predict marks for 11 study hours
new_data = pd.DataFrame([[11]], columns=["Hours"])
pred = model.predict(new_data)
print("Predicted Marks for 11 hours:",
      round(pred[0], 2))
```

Output:-

```
Training Accuracy: 99.73
Testing Accuracy: 99.76
Predicted Marks for 11 hours: 104.53
```

Q.3 (This Question must be included in your Practical 7 Part 2 Documentation. Take Print for this Question)

1. California Housing Dataset

Problem Statement

A real estate company wants to predict the median house value based on housing features. Median Income, House Age, Average Rooms, Average Bedrooms, Population Average Occupancy, Latitude, Longitude, Target, Median House Value

Tasks

- 1. Load the dataset using Pandas.**
- 2. Display the first five records.**
- 3. Check for missing values.**
- 4. Select one suitable feature (e.g., Median Income) as the independent variable.**
- 5. Split the dataset into training and testing sets.**
- 6. Train a Linear Regression model.**
- 7. Predict the house prices.**
- 8. Display: Coefficient, Intercept, R2 Score**
- 9. Plot the regression line.**
- 10. Interpret the results.**

Code:-

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Load Dataset
df = pd.read_csv("California_Housing.csv")
# Display first five records
print(df.head())
# Check Missing Values
print("\nMissing Values\n")
print(df.isnull().sum())
# Independent Variable
X = df[["MedInc"]]
# Dependent Variable
y = df["MedHouseVal"]
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train Model
model = LinearRegression()
model.fit(X_train, y_train)
```

```

# Prediction
prediction = model.predict(X_test)
print("\nCoefficient =", model.coef_[0])
print("Intercept =", model.intercept_)
print("R2 Score =", model.score(X_test,y_test))
# Graph
plt.scatter(X,y)
plt.plot(X,model.predict(X),color="red")
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("California Housing Dataset")
plt.show()

```

Output:-

```

... MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude \
0 8.3252 41 6.98 1.02 322 2.55 37.88
1 8.3014 21 6.24 0.97 2401 2.11 37.86
2 7.2574 52 8.28 1.07 496 2.80 37.85
3 5.6431 52 5.81 1.07 558 2.54 37.85
4 3.8462 52 6.28 1.08 565 2.18 37.85

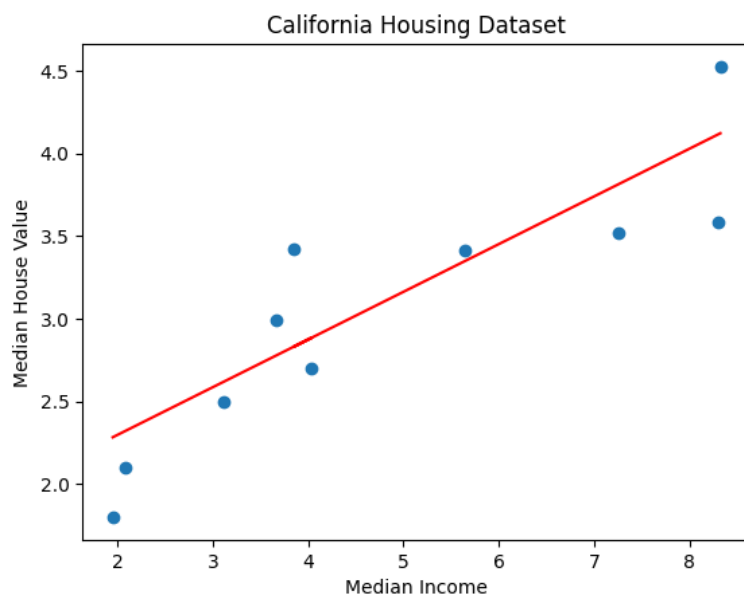
Longitude MedHouseVal
0 -122.23 4.526
1 -122.22 3.585
2 -122.24 3.521
3 -122.25 3.413
4 -122.25 3.422

Missing Values

MedInc 0
HouseAge 0
AveRooms 0
AveBedrms 0
Population 0
AveOccup 0
Latitude 0
Longitude 0
MedHouseVal 0
dtype: int64

Coefficient = 0.2885456269632562
Intercept = 1.720304716236585

```



3. Using the MNIST dataset (digit vs not digit), implement a binary classifier.

Code:-

```
# Load MNIST-like digits dataset
# Load dataset
data = pd.read_csv("/content/MNIST_Digits_Dataset.csv")
digits = load_digits()
X = digits.data
y = digits.target

# Binary classification
# True if digit is 5, otherwise False
y = (y == 5)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)

print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Predict first test sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 5")
else:
    print("Digit is not 5")
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 98.88888888888889
Digit is not 5
```

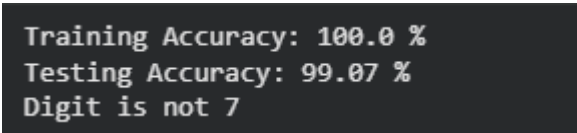
Questions on Part 3 (Include all the code in Print document)

Q.1 Modify the program to detect Digit 7 vs Not Digit 7 instead of Digit 5 vs Not Digit 5.

Code:-

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
# Load digits dataset
data = pd.read_csv("/content/MNIST_Digits_Dataset.csv")
digits = load_digits()
# Features and Target
X = digits.data
y = digits.target
# Binary classification
# True if digit is 7, otherwise False
y = (y == 7)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# Accuracy
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
# Predict first test sample
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 7")
else:
    print("Digit is not 7")
```

Output:-



```
Training Accuracy: 100.0 %
Testing Accuracy: 99.07 %
Digit is not 7
```

Q.2 Modify the program to detect Digit 0 vs Not Digit 0.

Code:-

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
# Load digits dataset
data = pd.read_csv("/content/MNIST_Digits_Dataset.csv")
digits = load_digits()
```

```

# Features and Target
X = digits.data
y = digits.target

# Binary Classification
# True if digit is 0, otherwise False
y = (y == 0)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Accuracy
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")

# Predict First Test Sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 0")
else:
    print("Digit is not 0")

```

Output:-

```

Training Accuracy: 100.0 %
Testing Accuracy: 99.44 %
Digit is not 0

```

Q.3 Modify the program to detect Digit 9 vs Not Digit 9.

Code:-

```

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load digits dataset
data = pd.read_csv("/content/MNIST_Digits_Dataset.csv")
digits = load_digits()
# Features and Target
X = digits.data
y = digits.target

```

```

# Binary Classification
# True if digit is 9, otherwise False
y = (y == 9)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Accuracy
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")

# Predict First Test Sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 9")
else:
    print("Digit is not 9")

```

Output:-

```

Training Accuracy: 99.68 %
Testing Accuracy: 98.52 %
Digit is not 9

```

Q.4 Modify the program to display the first 20 predictions instead of only one prediction.

Code:-

```

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load digits dataset
data = pd.read_csv("/content/MNIST_Digits_Dataset.csv")
digits = load_digits()

# Features and Target
X = digits.data
y = digits.target

# Binary Classification

```

```

# True if digit is 5, otherwise False
y = (y == 5)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Accuracy
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")

# Predict First 20 Test Samples
predictions = model.predict(X_test[:20])

print("\nFirst 20 Predictions:\n")

for i in range(20):
    if predictions[i]:
        print("Sample", i + 1, ": Digit is 5")
    else:
        print("Sample", i + 1, ": Digit is not 5")

```

Output:-

```

Training Accuracy: 100.0 %
Testing Accuracy: 98.89 %

First 20 Predictions:

Sample 1 : Digit is not 5
Sample 2 : Digit is 5
Sample 3 : Digit is not 5
Sample 4 : Digit is not 5
Sample 5 : Digit is not 5
Sample 6 : Digit is not 5
Sample 7 : Digit is not 5
Sample 8 : Digit is not 5
Sample 9 : Digit is 5
Sample 10 : Digit is not 5
Sample 11 : Digit is not 5
Sample 12 : Digit is not 5
Sample 13 : Digit is not 5
Sample 14 : Digit is not 5
Sample 15 : Digit is not 5
Sample 16 : Digit is not 5
Sample 17 : Digit is not 5
Sample 18 : Digit is not 5
Sample 19 : Digit is not 5
Sample 20 : Digit is not 5

```